Priority-Based Batch Job Scheduler

A greedy algorithm solution for online job scheduling under deadline constraints — DAA Capstone Project

NAME : BALI GOUTHAM NAIDU

CH.SC.U4CSE24064

Problem Statement

A batch processing engine receives a continuous stream of job requests, each with a specific **compute cost** and a strict **expiration deadline**. The system must select exactly **N jobs per day** — without any visibility into future arrivals.

No Look-Ahead

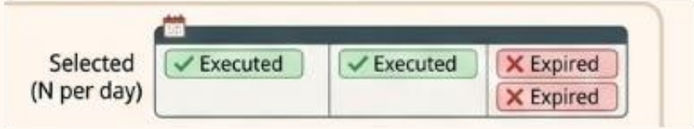
Future job characteristics are completely unknown at decision time.

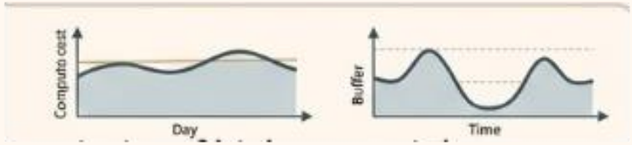
Hard Deadlines

A job is successful only if executed on or before its deadline day.

Load Balancing

Compute load must be smoothed across days for long-term system stability.





Objectives

What We Need to Solve:

The system receives a continuous stream of job requests, where each job requires a specific amount of computational effort and must be completed before a strict deadline.
The goal is to **select the best N jobs from today's queue** so that the scheduling decision remains effective even if challenging job requests arrive in the futur

Our Approach

To solve this problem, we use a **Greedy Scheduling Strategy** that prioritizes jobs based on their urgency and computational cost. The algorithm evaluates each job and selects the most suitable ones for execution.

Steps in Our Approach

➤ **Define a Priority Score**

Calculate a composite priority score for each job based on **deadline urgency** and **compute units required**.

➤ **Sort Jobs by Priority**

Arrange all pending jobs in descending order using a **greedy selection approach**.

➤ **Select Top N Jobs**

Choose the **top N highest priority jobs** for today's execution batch to ensure efficient and balanced scheduling.

Proposed Algorithm — Design

Algorithm: Greedy Priority Scheduling

Each job is assigned a **priority score** using a composite formula that rewards urgency (proximity to deadline) and efficiency (low compute cost):

$$P(J_i) = \frac{1}{(D_i - T_{today}) + 1} + \frac{1}{C_i}$$

Where **D_i** is the deadline, **T_today** is the current day, and **C_i** is the compute cost. Jobs with imminent deadlines or low compute costs rank higher.

Urgency Term

`1 / (deadline - today + 1)`Larger when deadline is near

Efficiency Term

`1 / compute`Larger for lightweight jobs

Combined Score

Sum of both terms — balances deadline risk with resource cost

Proposed Algorithm — Pseudocode

```
INPUT: jobs[k], N, today

FUNCTION calculatePriority(compute, deadline):
    urgency = deadline - today
    IF urgency < 0 THEN urgency = 0
    RETURN (1.0 / (urgency + 1)) + (1.0 / compute)

FOR each job J_i in jobs[k]:
    J_i.priority = calculatePriority(J_i.compute,
                                     J_i.deadline)

SORT jobs[] in DESCENDING order of priority
    (using qsort with custom comparator)

OUTPUT: jobs[0] to jobs[N-1]
    (Top-N highest priority jobs)
```

Key Design Decisions

- Urgency is floored at `0` to handle already-expired jobs gracefully
- `qsort` from `stdlib.h` provides an efficient, in-place sort with **O(k log k)** average time
- No future data is ever read — the algorithm is fully **online and causal**
- Selection is a simple top-N slice — deterministic and reproducible

Algorithm Complexity

Worst-Case Time

When pivot selection is poor in `qsort`

Average-Case Time

Priority calculation O(k) + sorting O(k log k)

Space Complexity

In-place sort, no auxiliary data structures needed

The dominant cost is the **sort step**. Priority calculation is a single linear pass O(k). The final selection pass (top-N) is O(N). For practical batch sizes, this is highly efficient.

Implementation

```c
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int id, compute, deadline;
    float priority;
} Job;

int today = 1;

float calculatePriority(int compute, int deadline) {
    int urgency = deadline - today;
    if (urgency < 0) urgency = 0;
    return (1.0 / (urgency + 1)) + (1.0 / compute);
}

int compare(const void *a, const void *b) {
    Job *j1 = (Job *)a, *j2 = (Job *)b;
    if (j1->priority < j2->priority) return 1;
    if (j1->priority > j2->priority) return -1;
    return 0;
}
```

Language & Tools

C (C99)

Core implementation language

stdlib.h

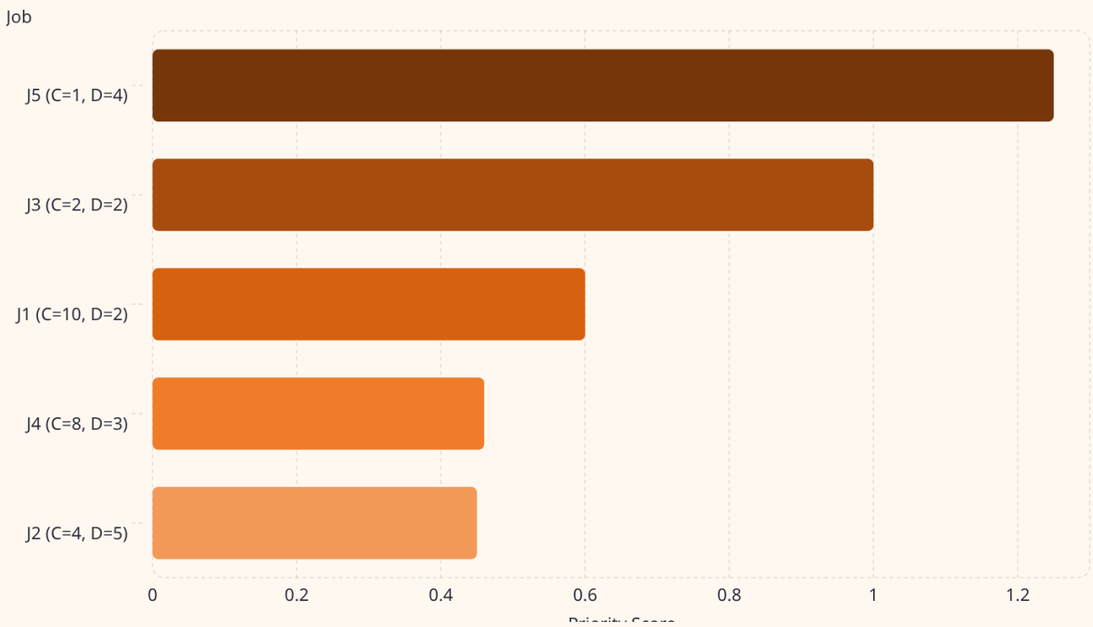
`qsort()` for in-place sorting

stdio.h

Interactive I/O via `scanf` / `printf`

VLA `Job jobs[k]` used for dynamic input size — compile with C99 or later.

Results & Analysis



Sample Run — 5 Jobs, N=3

Input: 5 jobs with varying compute and deadlines. Today = Day 1. Select N = 3.

Output — Top 3 Scheduled:

J3 and J1 are correctly elevated due to their **same-day deadlines**, while J5 ranks first for its near-zero compute cost.

J5 ✓

Compute=1, Deadline=4
Priority: **1.25**

J3 ✓

Compute=2, Deadline=2
Priority: **1.00**

J1 ✓

Compute=10, Deadline=2
Priority: **0.60**

Conclusion

1

Effective Online Scheduling

The greedy priority formula balances urgency and efficiency without requiring future knowledge — making it a practical online algorithm.

2

Scalable & Efficient

O(k log k) average time complexity ensures the scheduler remains fast even as the job queue grows.

3

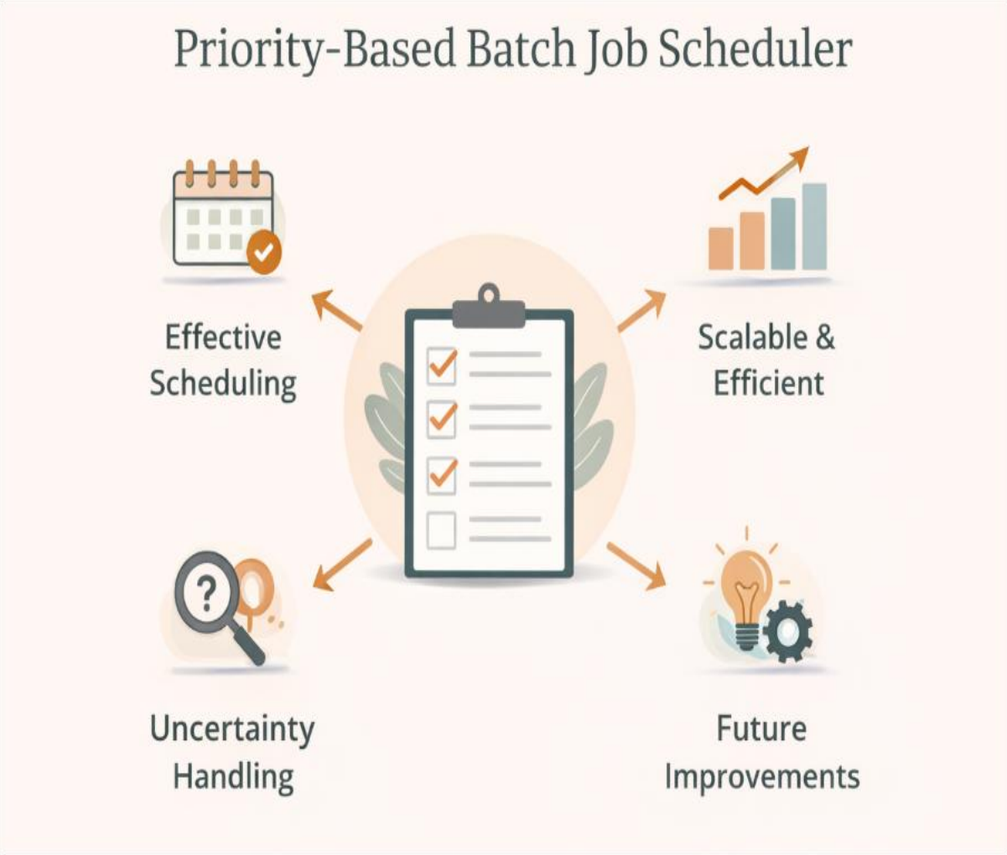
Robust Under Uncertainty

By prioritizing near-deadline and low-cost jobs, today's selection is defensible regardless of what future jobs arrive.

4

Future Work

Weighted priority tuning, machine-learning-assisted urgency prediction, or multi-resource constraint scheduling could further improve outcomes.



Thank You

Priority-Based Batch Job Scheduler — DAA Capstone Project

CH.SC.U4CSE24064